

# CSC443 – Spring 2026

## Project Phase 2

Due Date: 20/04/2026

### Phase 2: Full-Stack Application Integration (Final Project)

#### Project Brief:

In this final phase, your team will build upon the frontend developed in Phase 1 by creating a robust backend API using Node.js and Express.js, integrating it with a database (MySQL/PostgreSQL/MongoDB), and implementing secure user authentication and authorization. The goal is to transform your simulated frontend into a fully functional, persistent, and secure full-stack web application, based on your phase 1 **topic**.

#### Team & Project Continuity:

- Continue with the same team and the web application project from Phase 1.
- You will integrate your existing frontend with the new backend.

#### Core Technologies to Utilize:

- **Backend:** Node.js, Express.js.
- **Database:** MySQL, PostgreSQL or MongoDB (using a Node.js client library like mysql2, pg, mongodb or mongoose).
- **Authentication:** JSON Web Tokens (JWTs), bcrypt for password hashing.
- **Deployment:** Cloud services (e.g., Render, Cyclic, Heroku for backend; Netlify... for frontend).
- **Version Control:** Git & GitHub (continuing from Phase 1, or a separate backend repo, but integration will be key).

#### Project Requirements (Phase 2):

1. **Backend API Development with Node.js & Express:**
  - **RESTful API:** Design and implement a clear and consistent RESTful API using Node.js and Express.js for managing your chosen data entities and users.
  - **Endpoints:** Create API routes for:
    - User registration and login.
    - Retrieving all of your primary data entities.
    - Retrieving a single data entity by ID.
    - Creating a new data entity (authenticated).
    - Updating an existing data entity by ID (authenticated, user must be owner/authorized).
    - Deleting a data entity by ID (authenticated, user must be owner/authorized).
  - **Middleware:** Utilize Express middleware for request parsing (JSON), logging, and robust error handling.
2. **Database Design & Integration:**
  - **Schema Design:** Design and implement a logical database schema for users and your primary data entities, including appropriate relationships (e.g., a data entity belongs to a user). Define relevant fields for your topic.
  - **Connection:** Connect your Express API to a hosted database.

- **CRUD Operations:** Implement full Create, Read, Update, and Delete (CRUD) operations on your chosen database for both users and your primary data entities via your API endpoints.
- 3. **User Authentication & Authorization:**
  - **User Registration:** Allow new users to register with a username/email and password.
  - **Password Security:** Store user passwords securely by hashing them using bcrypt (or a similar library) before saving them to the database.
  - **User Login:** Implement user login functionality that authenticates credentials against the hashed passwords.
  - **Token-Based Authentication:** Upon successful login, issue a JSON Web Token (JWT) to the user.
  - **Protected Routes:** Implement middleware to protect specific API routes (e.g., creating, updating, deleting data entities) so that only authenticated users with a valid JWT can access them.
  - **Authorization:** Ensure users can only modify or delete data entities they have personally created (or are otherwise authorized for, based on your topic's logic).
- 4. **Full-Stack Integration:**
  - **API Consumption:** Modify your React frontend from Phase 1 to consume data from your custom-built Node.js/Express API. This involves replacing all mock data interactions with actual fetch API calls (or axios) to your backend.
  - **State Management:** Update your React components to manage application state based on data retrieved from the backend.
  - **User Feedback:** Provide appropriate user feedback for API calls (e.g., loading indicators, success/error messages).
- 5. **Deployment:**
  - **Backend Deployment:** Successfully deploy your Node.js/Express backend API to a cloud service (e.g., Render, Cyclic, Heroku).
  - **Frontend Deployment:** Ensure your frontend application from Phase 1 is still deployed and correctly configured to communicate with your deployed backend API. The entire application must be functional and accessible online.
- 6. **Code Quality & Documentation:**
  - Maintain high code quality for both frontend and backend.
  - Add comprehensive comments where needed, especially for complex logic or API routes.

### **Submission Criteria (Phase 2 - Final Project):**

1. **Deployed Full-Stack Application Links:**
  - A link to the live, deployed frontend application.
  - A link to the live, deployed backend API.
2. **GitHub Repository Link(s):**
  - Link(s) to your public GitHub repository/repositories (e.g., a single monorepo, or separate frontend/backend repos).
3. **Comprehensive README.md File:** A detailed README.md file in your *main* project repository (or linked from both if separate) that includes:
  - Project Title and Team Member Names.
  - The randomly assigned topic and the primary data entities chosen.
  - Links to the deployed frontend and backend.

- Detailed setup instructions for running the **entire full-stack project locally** (frontend and backend).
- **Comprehensive API documentation:** List all backend endpoints, HTTP methods, expected request/response bodies, and authentication requirements.
- Screenshots or a GIF showcasing the final, fully functional application.
- A brief overview of each team member's primary contributions to both phases of the project.
- Any significant technical challenges encountered and how they were overcome.

Good luck!